

SOFTWARE DESIGN DOCUMENT (SDD)

Smart Order and Inventory Management System (OMS)

1. DOCUMENT CONTROL

Item	Details
Project Name	Smart Order Management System
Version	1.0
Author	(Your Name)
Date	(Date)
Status	Final

2. PURPOSE OF THE DOCUMENT

This document describes the **system architecture, design decisions, component structure, APIs, data models, and non-functional aspects** of the Smart Order Management System.

It is intended for:

- Developers
 - Reviewers
 - Interview discussions
 - Maintenance & enhancement planning
-

3. SYSTEM OVERVIEW

Business Objective

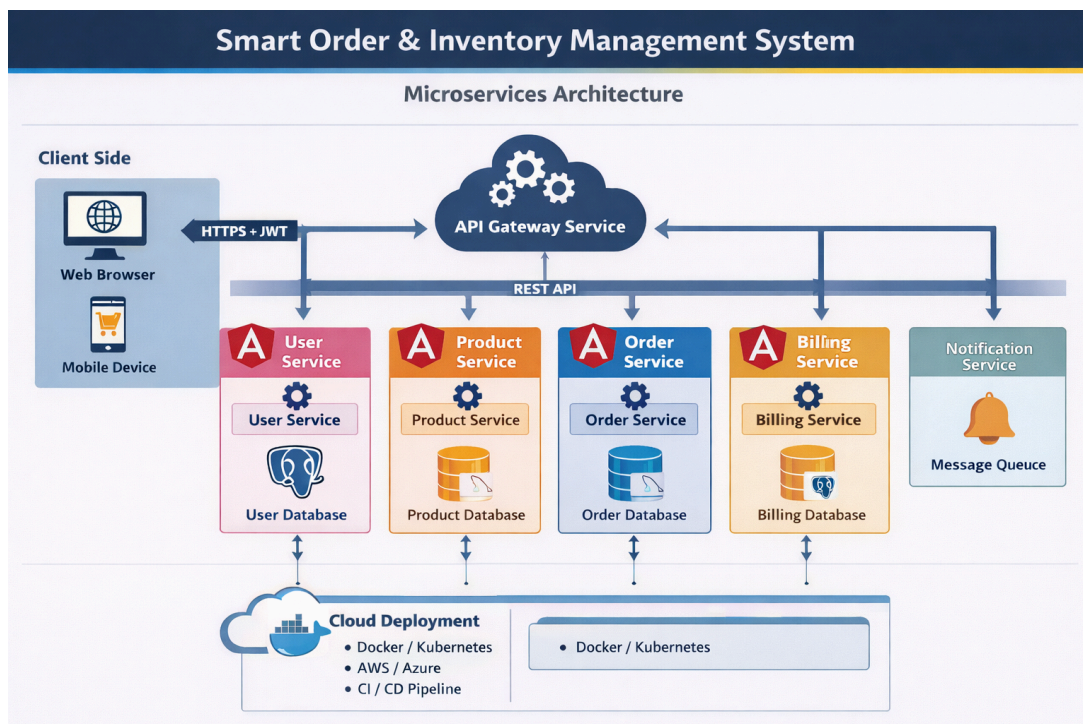
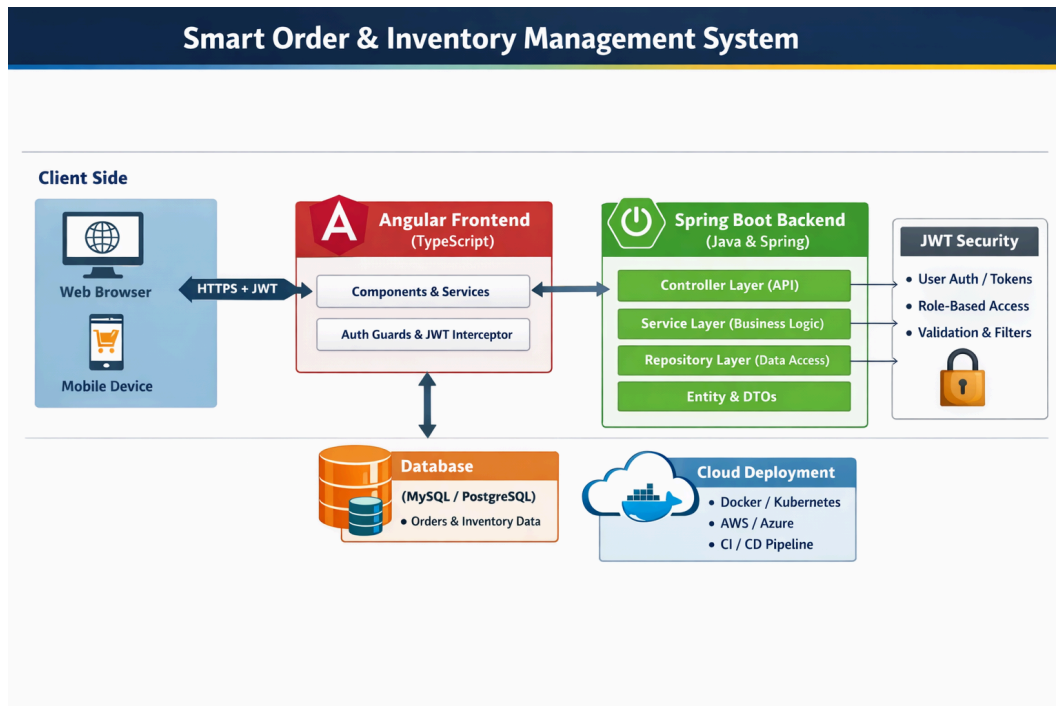
Provide a scalable, secure, and maintainable system to:

- Manage users
- Manage products
- Place and track orders
- Support growth through microservices

High-Level Features

- User registration & login
- Product catalog management
- Order placement & tracking
- Role-based access (USER / ADMIN)

4. ARCHITECTURE OVERVIEW (HLD)



Architecture Style

1. Microservices Architecture
2. REST-based communication
3. JWT-based security
4. Containerized deployment (Docker)
5. Scalable and loosely coupled design

Core Components

1. Angular Frontend
 2. API Gateway (for routing and security – optional)
 3. User Service (Authentication & Authorization)
 4. Product Service (Product catalog management)
 5. Order Service (Order placement & tracking)
 6. Billing Service (Invoice & payment handling)
 7. Notification Service (Email / system notifications)
 8. MongoDB (Dedicated database per microservice)
 9. CI/CD Pipeline (Automated build, test, and deployment)
-

5. TECHNOLOGY STACK

Backend

- Java 17
- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Validation

Frontend

- Angular
- Angular Material
- RxJS

Database

- MySQL(one DB per microservice)

DevOps

- Docker
- Docker Compose
- Jenkins
- SonarQube

Testing

- JUnit 5
 - Mockito
 - Spring Boot Test
 - JMC
 - JMeter
-

6. MICROSERVICES DESIGN

6.1 Auth Service

Responsibilities

- User registration
- Authentication
- Role management

APIs

METHOD	ENDPOINT	REQUEST PAYLOAD	RESPONSE PAYLOAD
POST	/auth/register	{ name, email, password }	-
POST	/auth/login	{ email, password }	{ name,email, role } Token in cookie
POST	/auth/logout	-	-

Database

- users database
-

6.2 Product Service

Responsibilities

- Product CRUD
- Inventory management

APIs

METHOD	ENDPOINT	REQUEST PAYLOAD	RESPONSE PAYLOAD
POST	/products	{ "name": "iPhone 12 Pro", "description": "Apple Phone",	{ "id": 6, "name": "iPhone 9 Pro",

		<pre>"brand": "Apple", "price": 100 }</pre>	<pre>"description": "Apple phone", "brand": "Apple", "price": 109999 }</pre>
GET	/products	—	<pre>[{ "id": 1, "name": "MacBook Air M3", "description": "Apple laptop", "brand": "Apple", "price": 129999.00 }]</pre>
PUT	/products/{id}	<pre>{ "price": 899.00, "specification": { "colors": ["Black", "Grey"], } }</pre>	200 OK
DELETE	/products/{id}	—	200 OK
GET	/products/{id}	-	<pre>{ "id": 1, "name": "MacBook Air M3", "description": "Apple laptop", "brand": "Apple", "price": 129999.00 }</pre>
GET	/products/{name}	—	<pre>{ "name": "Wireless Mouse", "description": "Ergonomic wireless mouse", "brand": "Logitech", "specification": { "colors": ["Black", "White", "Blue"], ... } "price": 799.00, "stock": 150 }</pre>
PATCH	/products/{id}/stock	<pre>{ "quantity": 30, "operation": "ADD" }</pre>	200 OK

GET	/products/{id}/stock	-	{ "id": "P001", "stock": 50 }
-----	----------------------	---	--

Database

- products collection

6.3 Order Service

Responsibilities

- Order creation
- Order history
- Order status

APIs

<u>METHOD</u>	<u>ENDPOINT</u>	<u>REQUEST PAYLOAD</u>	<u>RESPONSE PAYLOAD</u>
POST	/orders	{ "customerId": 12, "items": [{ "productId": 101, "quantity": 2 }, {"productId": 205, "quantity": 1 }] }	{ "orderId": 501, "status": "CREATED", "totalAmount": 3499.00 }
GET	/orders/{orderId}	—	{ "orderId": 501, "status": "PACKED", "items": [{ "productId": 101, "quantity": 2, "price": 999 }, { "productId": 205, "quantity": 1, "price": 1501 }], "totalAmount": 3499, "warehouse": "BLR-WH-01" }

GET	/orders/user/{userId}?page=0&size=10	—	{ "content": [{ "orderId": 501, "status": "DELIVERED", "totalAmount": 3499, "createdAt": "2025-12-27T10:45:12" }, { "orderId": 498, "status": "CANCELLED", "totalAmount": 1299, "createdAt": "2025-12-25T18:30:00" }], "totalPages": 3, "totalElements": 21 }
GET	/orders/{id}/status	—	[{ "status": "CREATED", "changedAt": "2025-12-27T10:45:12", "changedBy": "customer" }, { "status": "APPROVED", "changedAt": "2025-12-27T11:10:00", "changedBy": "admin" }, { "status": "PACKED", "changedAt": "2025-12-27T12:40:00", "changedBy": "warehouse_manager" }]
PUT	/orders/{id}/status	json { "status": "PACKED" }	{ "orderId": 501, "oldStatus": "APPROVED", "newStatus": "PACKED", "updatedAt": "2025-12-27T12:40:00" }
PUT	/orders/cancel/{id}	—	{ "orderId": 501, "status": "CANCELLED", "message": "Order cancelled successfully" }

Database

- orders database

6.4 Billing Service

METHOD	ENDPOINT	REQUEST PAYLOAD	RESPONSE PAYLOAD
--------	----------	-----------------	------------------

POST	/billing/invoice	{ orderId }	{ invoiceId, orderId, amount, status, createdAt }
GET	/billing/invoice/{orderId}	—	{ invoiceId, orderId, amount, tax, totalAmount, status }
PUT	/billing/payment/{invoiceId}	{ paymentMode }	{ invoiceId, paymentStatus, paidAt }
GET	/billing/status/{orderId}	—	{ orderId, invoiceId, paymentStatus }

7. DATA DESIGN (LLD)

User Document

```
{
  "id": "u101",
  "email": "user@test.com",
  "password": "encrypted",
  "role": "USER",
  "active": true
}
```

Product Document

```
{
  "id": "p201",
  "name": "Phone",
  "price": 30000,
  "quantity": 15,
  "status": "AVAILABLE"
}
```

Order Document

```
{
  "orderId": "o501",
  "userId": "u101",
  "items": [
    { "productId": "p201", "qty": 1 }
  ],
  "totalAmount": 30000,
}
```

```
"status": "PLACED"
}
```

8. API DESIGN & VALIDATION

- RESTful principles
 - JSON request/response
 - Bean Validation at DTO layer
 - Service-level business rule enforcement
 - Standard HTTP status codes
-

9. ERROR HANDLING STRATEGY

Global Exception Handling

- Centralized using `@ControllerAdvice`
- Standard error response format

```
{
  "timestamp": "2025-01-01T10:00:00",
  "status": 400,
  "error": "Validation Error",
  "message": "Price must be greater than zero"
}
```

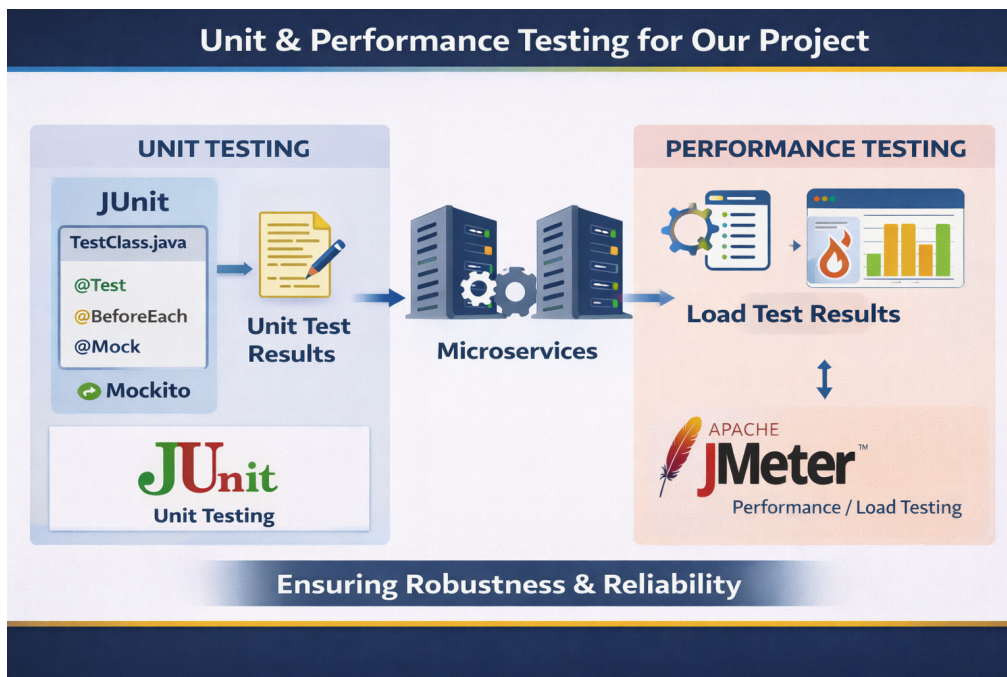
10. SECURITY DESIGN

- Password encryption (BCrypt)
 - Role-based access control
 - JWT authentication (optional enhancement)
 - Secure API access
-

11. NON-FUNCTIONAL REQUIREMENTS

Area	Design Decision
Scalability	Stateless services, Docker
Performance	Caching, async calls
Availability	Independent services
Maintainability	POM-like layered backend
Security	Validation, encryption
Observability	Logging & monitoring

12. TESTING STRATEGY



Backend

- Unit tests (Service layer)
- Controller tests (MockMvc)
- Minimum 90% coverage

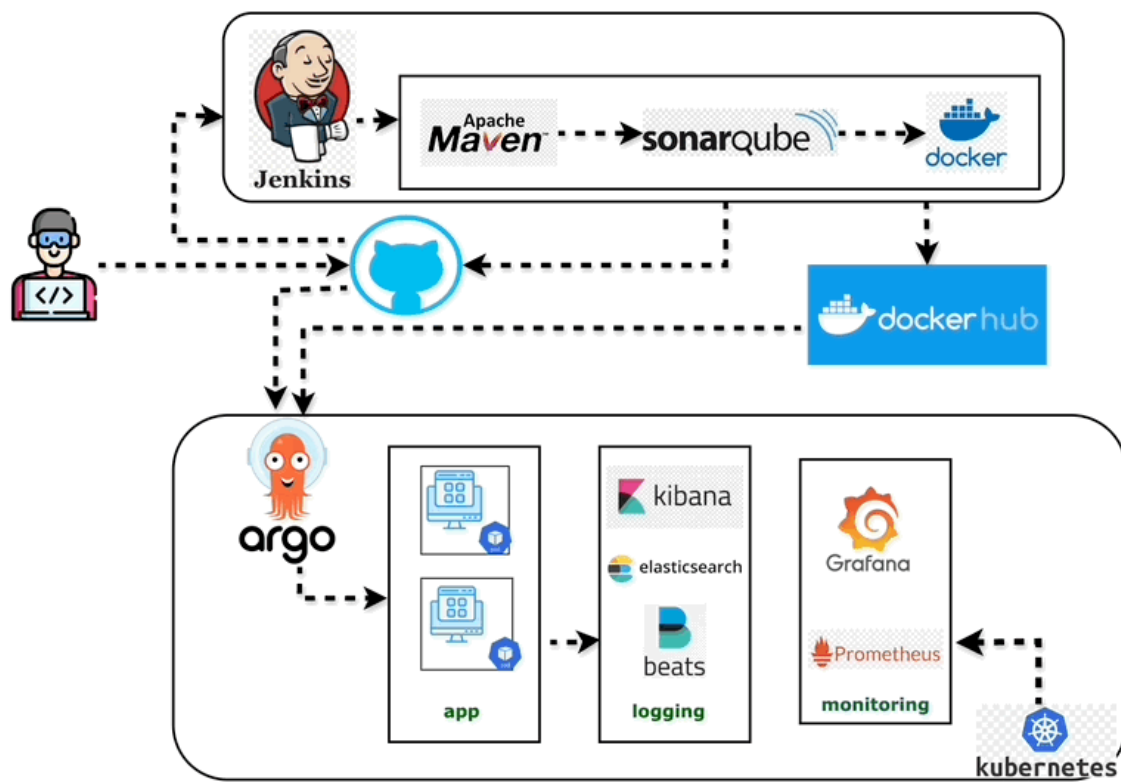
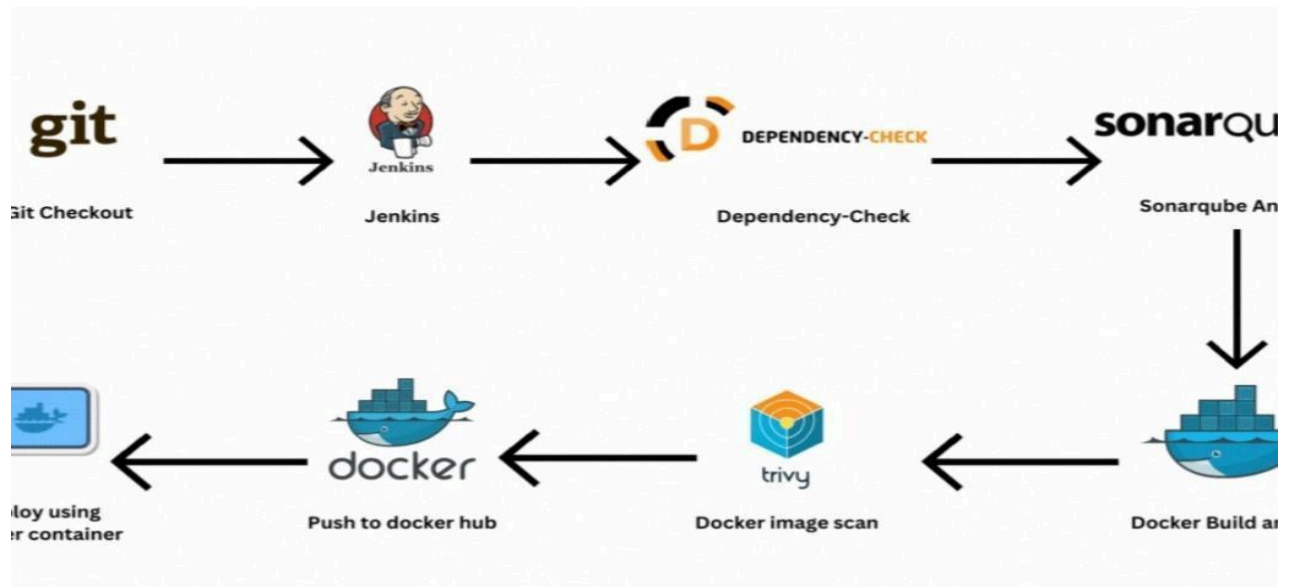
Frontend

- Component tests
- Service tests

Quality Gates

- SonarQube enforced
- Build fails on violations

13. CI/CD DESIGN



Pipeline Flow

1. Git Commit
2. Jenkins Build
3. Unit Tests
4. SonarQube Scan
5. Quality Gate Check
6. Docker Build
7. Docker Compose Deploy

14. DEPLOYMENT DESIGN

- Docker image per microservice
 - docker-compose for orchestration
 - Environment-specific configs
-

15. ASSUMPTIONS & CONSTRAINTS

Assumptions

- Services communicate over REST
- MongoDB available
- Docker environment present

Constraints

- No distributed transactions
 - Event-driven architecture out of scope
-

16. FUTURE ENHANCEMENTS

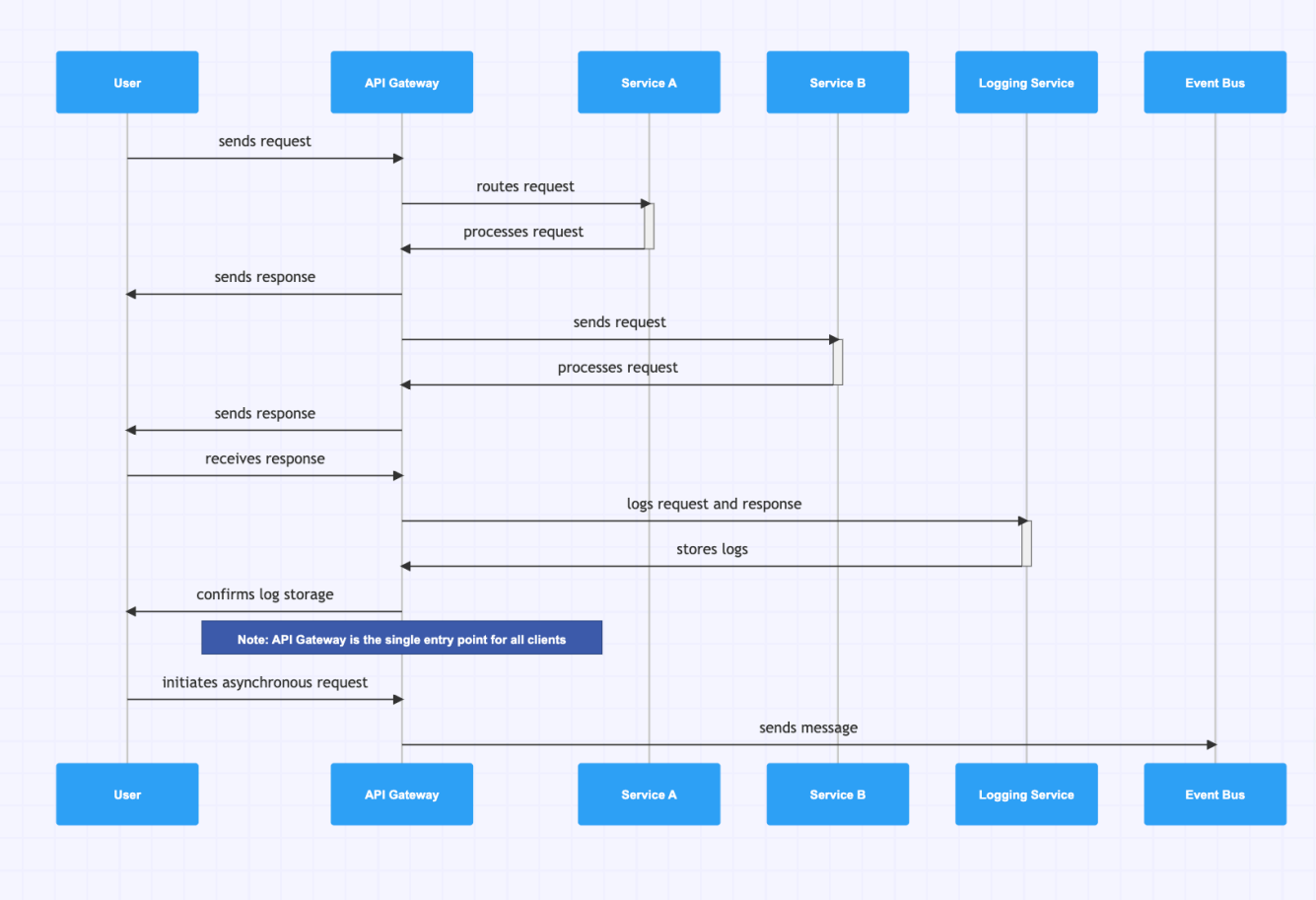
- Spring Cloud Gateway
 - Kafka-based async communication
 - Kubernetes deployment
 - Centralized logging (ELK)
-

17. CONCLUSION

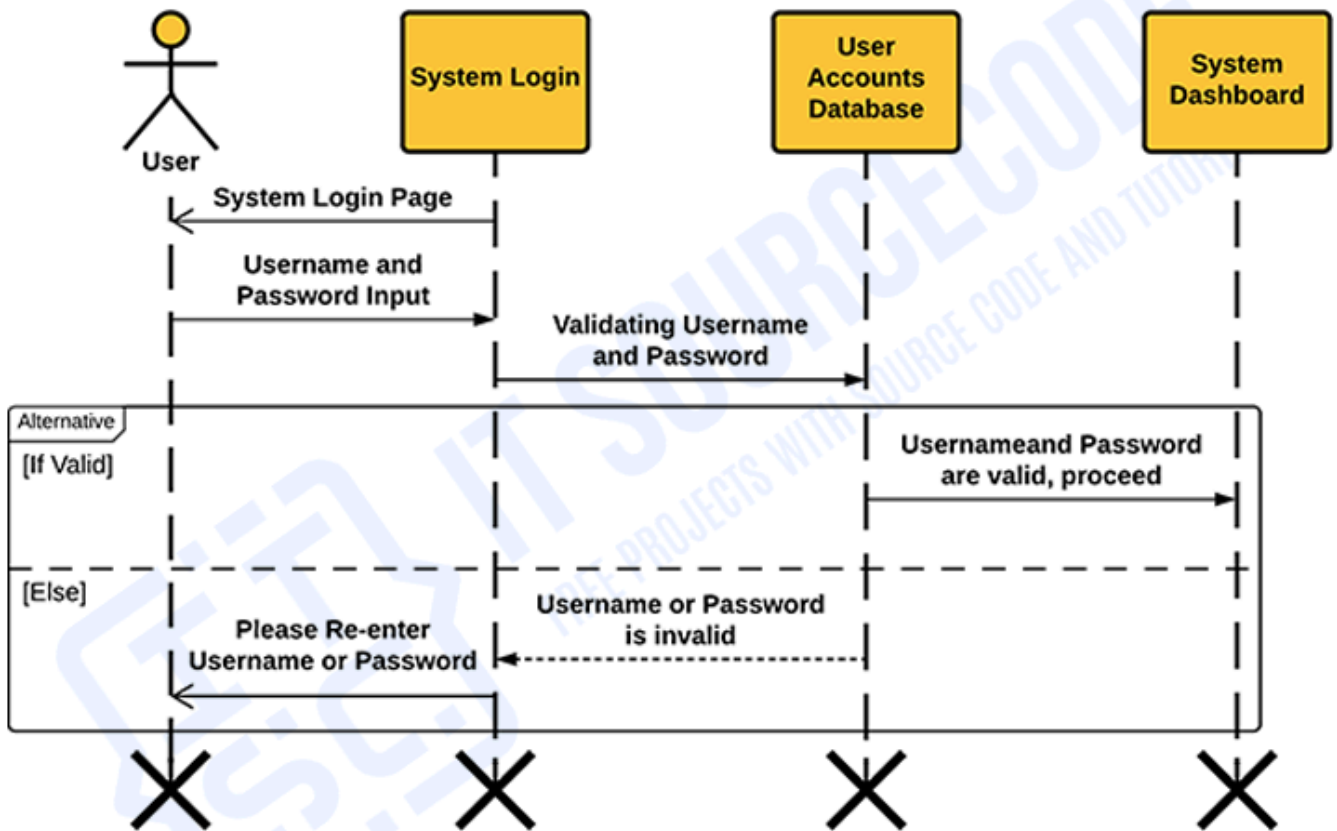
This design ensures:

- ✓ Clean separation of concerns
- ✓ Scalability & maintainability
- ✓ Testability & CI/CD readiness
- ✓ Interview-ready explanation

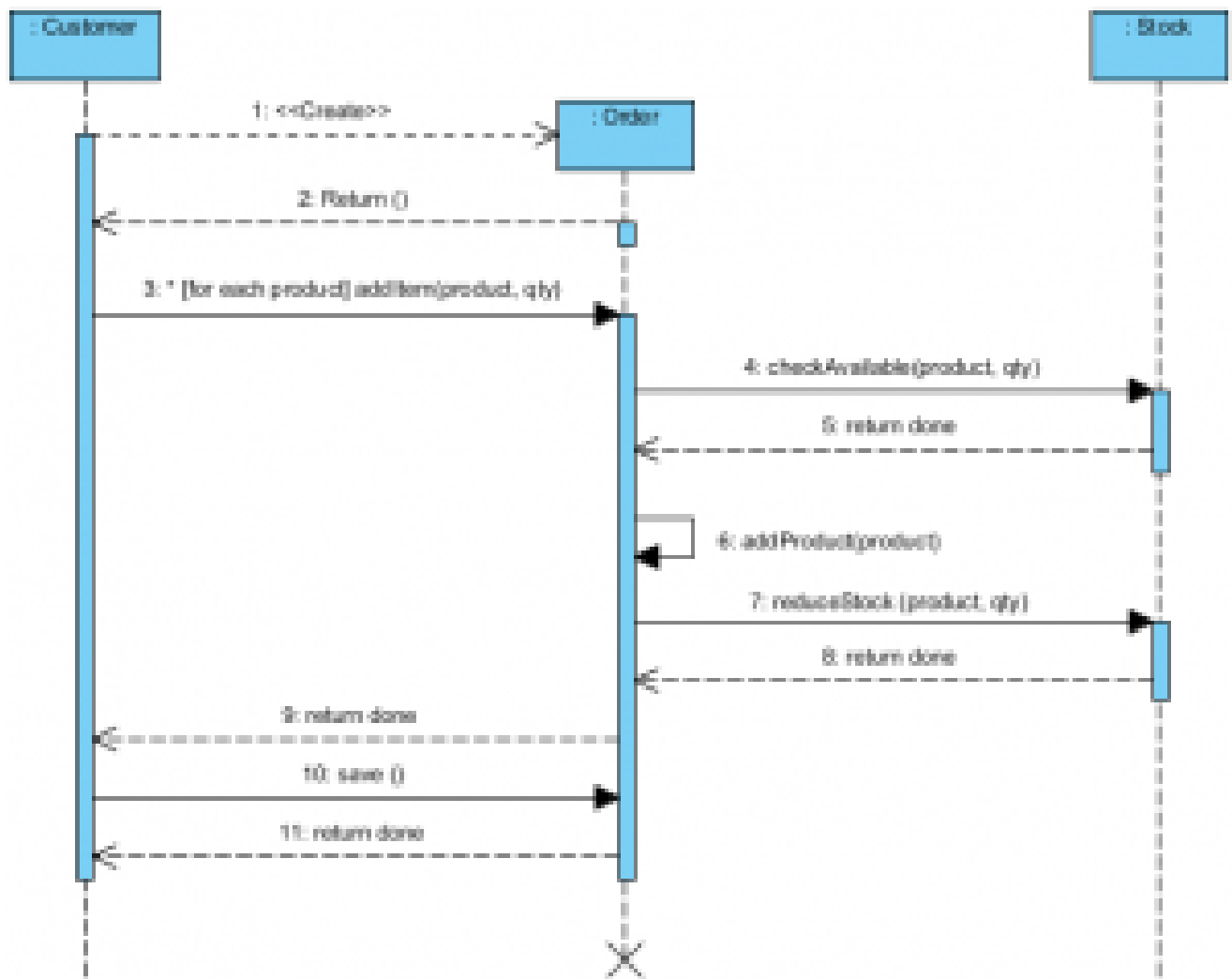
SEQUENCE DIAGRAMS — SMART ORDER MANAGEMENT SYSTEM



LOGIN SYSTEM



SEQUENCE DIAGRAM



1. User Registration — Sequence Diagram

Scenario

A new user registers using the Angular UI.

Flow

1. User enters details in Angular UI
2. Angular sends `POST /users/register` to User Service
3. User Service validates request (email, password)
4. User Service checks email uniqueness in MongoDB
5. Password is encrypted
6. User is saved in MongoDB
7. Success response sent back to UI

Participants

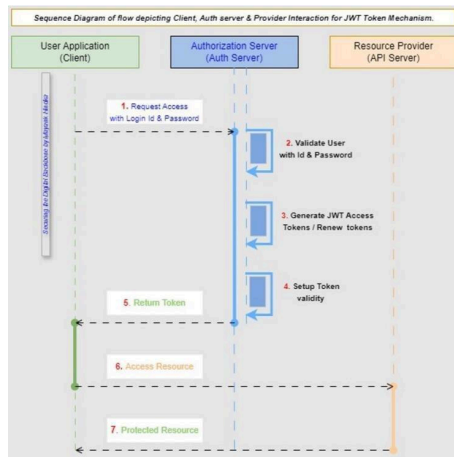
- User
- Angular UI

- User Service
- MongoDB

Key Design Points

- Validation at API boundary
 - Encryption at service layer
 - No direct DB access from UI
-

2. User Login — Sequence Diagram



Scenario

Registered user logs into the system.

Flow

1. User submits login form
2. Angular calls `POST /users/login`
3. User Service fetches user from MongoDB
4. Password is verified
5. (Optional) JWT token generated
6. Response sent to Angular
7. Angular stores token and navigates user

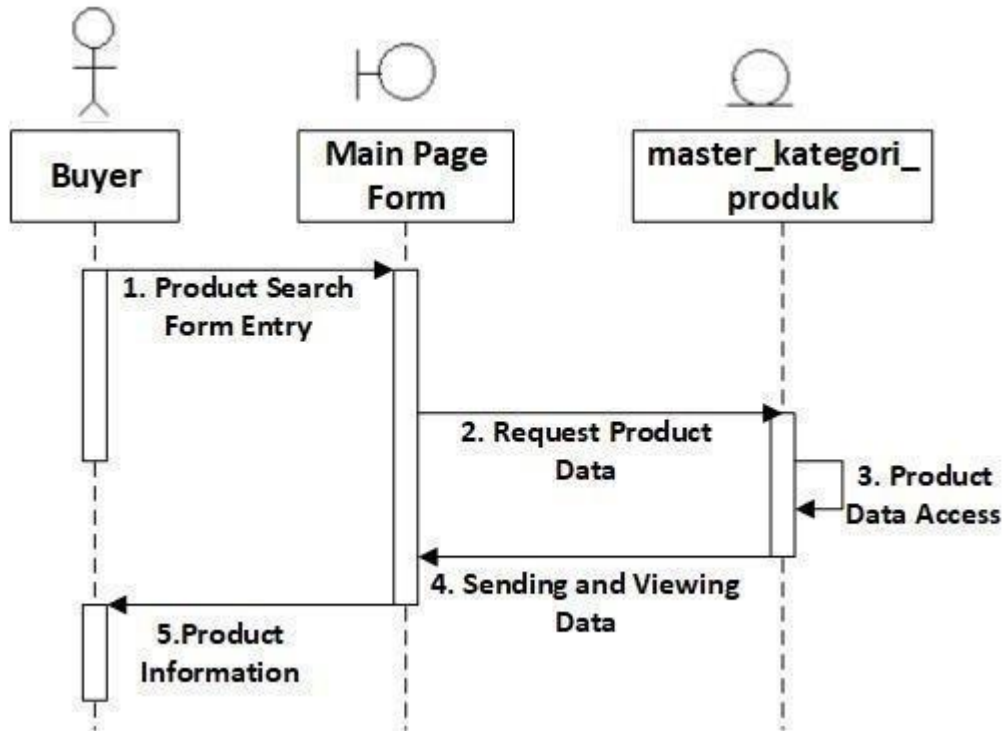
Participants

- User
- Angular UI
- User Service
- MongoDB

Failure Cases

- Invalid credentials → 401
- Inactive user → 403

3. Product Listing — Sequence Diagram



The screenshot shows a development environment with VS Code on the left and a web browser on the right. The VS Code editor displays the `people.service.ts` file, which defines the `PeopleService` class. The class is decorated with `@Injectable()` and imports `HttpClient` from `'@angular/common/http'`. The constructor takes a private `http: HttpClient` as a parameter. The web browser shows the `LearnHttpInAngular` application running on `localhost:4200`. The browser's main content area displays the text "Fetch data from some API".

Scenario

User views available products.

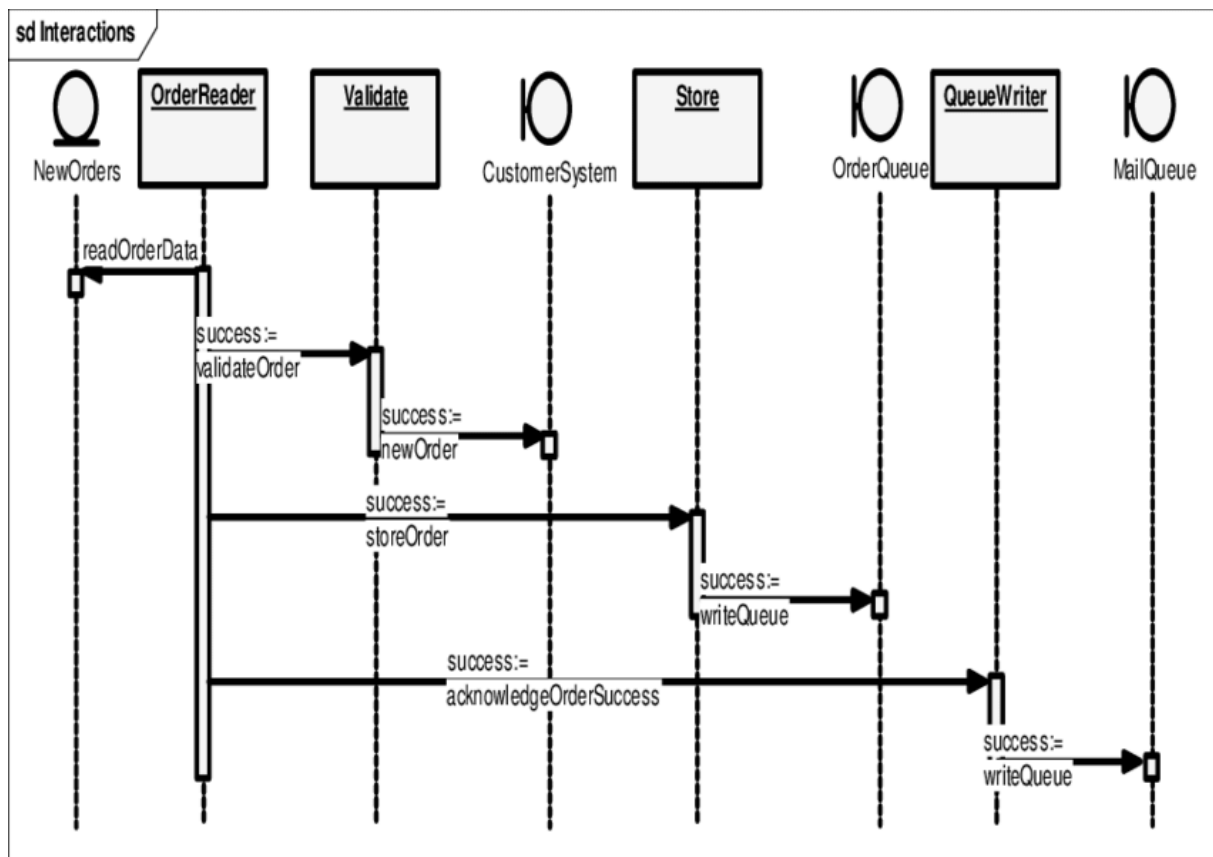
Flow

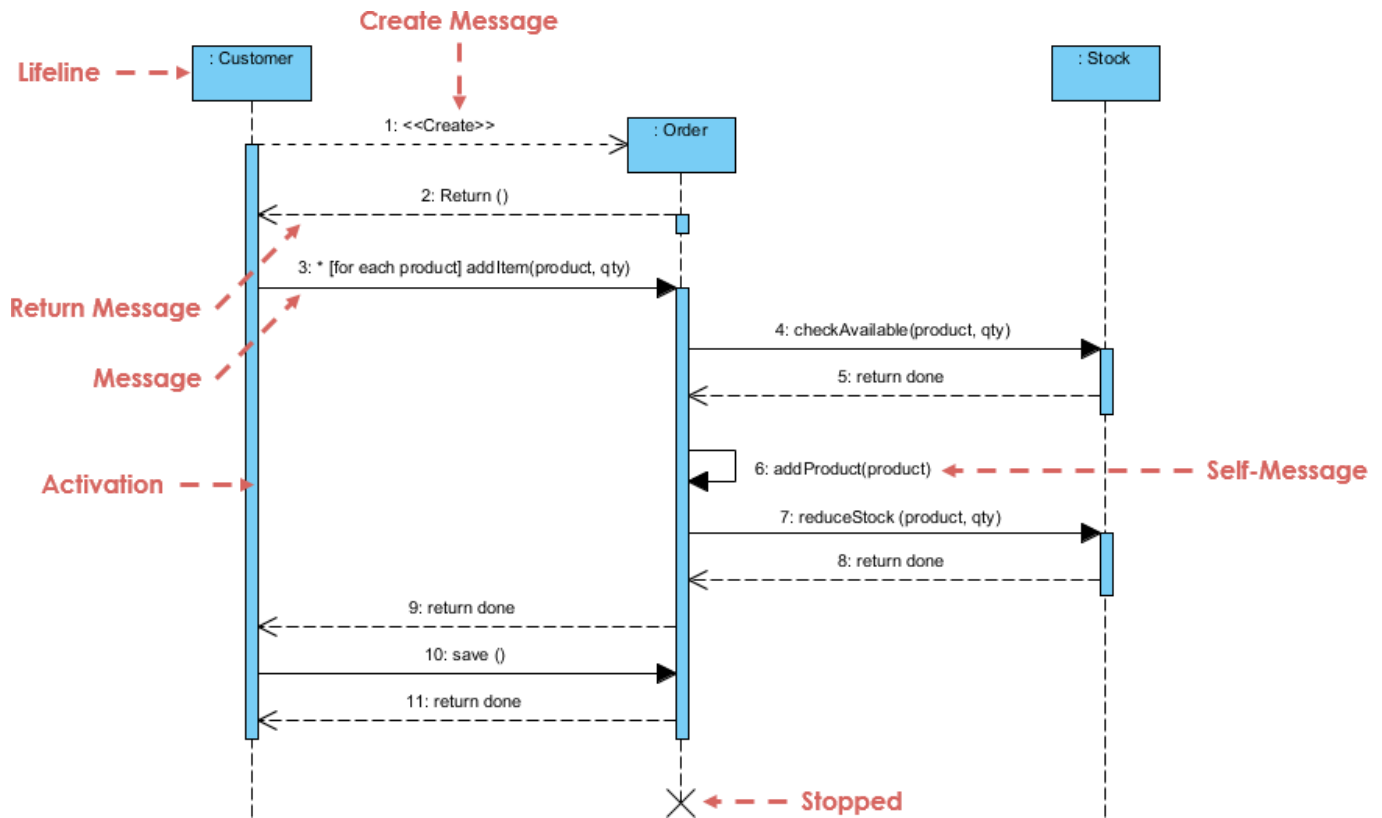
1. Angular loads product page
2. GET /products sent to Product Service
3. Product Service queries MongoDB
4. Product list returned to Angular

Participants

- Angular UI
- Product Service
- MongoDB

4. Order Placement — Sequence Diagram (MOST IMPORTANT)





Scenario

User places an order for one or more products.

Flow

1. User clicks **Place Order** in Angular
2. Angular sends `POST /orders` to Order Service
3. Order Service validates request
4. Order Service calls Product Service to check stock
5. Product Service validates availability
6. Order Service creates order
7. Product Service reduces inventory
8. Order saved in MongoDB
9. Success response returned to Angular

Participants

- User
- Angular UI
- Order Service
- Product Service
- MongoDB

Key Design Decisions

- Inventory validation before order creation
- Atomic stock update logic
- Failure stops order creation

5. Order Status Update (Admin) — Sequence Diagram

Store Dashboard Content Management Site Building Site Configuration Store Administration User Management Reports								0 / 1	Log Out Admin
Home > Administer > Store administration > Orders >									
Orders									
View order number:		View by status:		Action: *					
		Payment received		✓ - Please Select -		Go			
				Update Order Status					
				Update Tracking Number					
☐	Actions	Order ID	Custom	Purchase date	Payment date	Status			
☒	   	45	First Name / Last Name	S\$ 26.91	2013-05-28 14:10	2013-05-28 14:11	Payment received		
☒	   	44	First Name / Last Name	S\$ 26.91	2013-05-27 15:31	2013-05-27 16:11	Payment received		
☐	   	36	Ethan Cheong	S\$ 26.91	2013-04-26 17:22		Payment received		

Scenario

Admin updates order status.

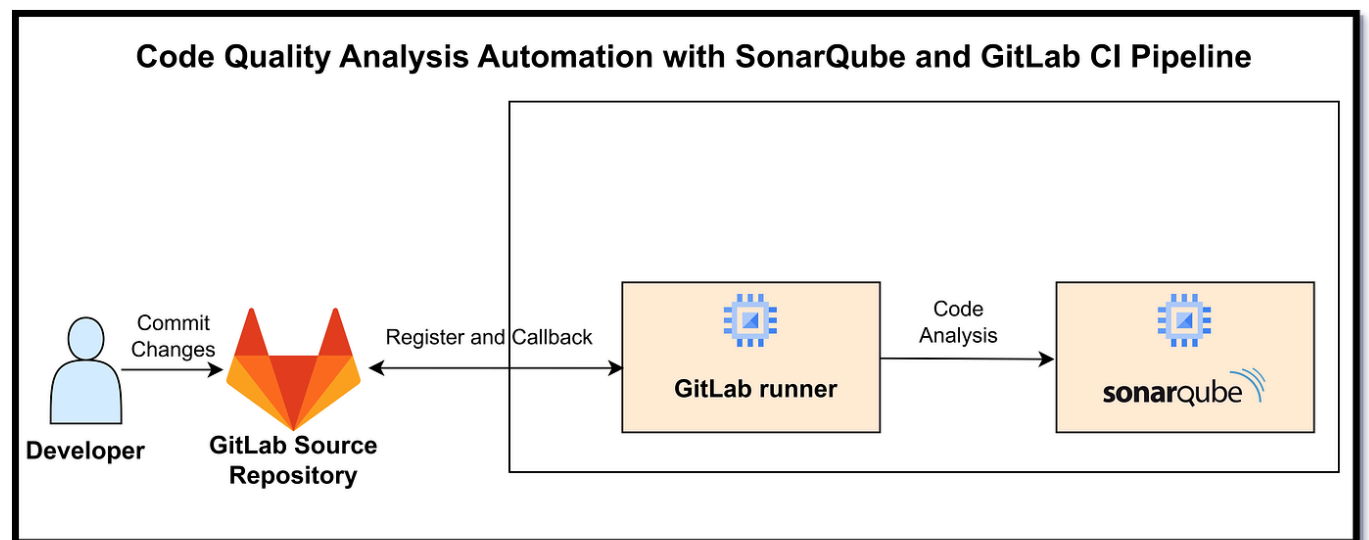
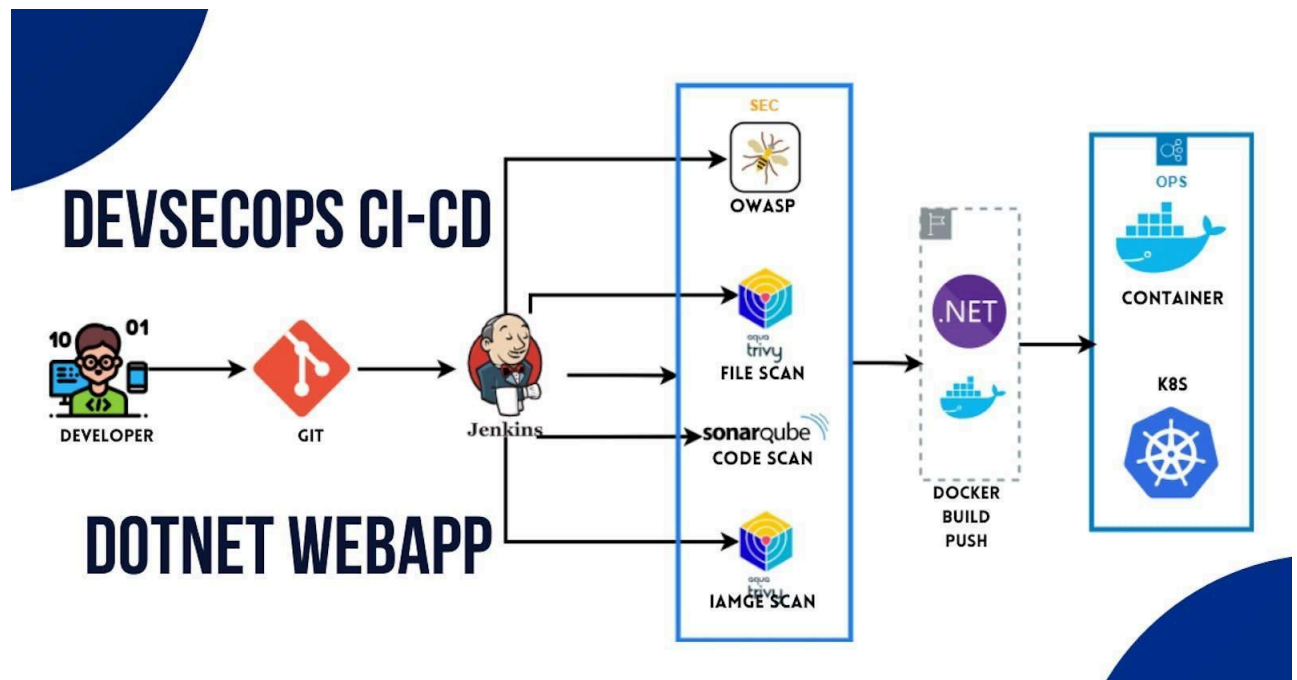
Flow

1. Admin sends update request from Angular
2. `PUT /orders/{id}/status` sent to Order Service
3. Role validation (ADMIN only)
4. Order status transition validated
5. Order updated in MongoDB
6. Updated status returned

Valid Transitions

- CREATED → PLACED
- PLACED → COMPLETED
- PLACED → CANCELLED

6. CI/CD Pipeline — Sequence Diagram



Scenario

Developer pushes code to Git repository.

Flow

1. Developer pushes code
2. Jenkins pipeline triggered
3. Jenkins runs unit tests
4. Jenkins runs SonarQube scan
5. Quality gate checked
6. Docker images built
7. Docker Compose deploys services

Participants

- Developer
 - Git
 - Jenkins
 - SonarQube
 - Docker
-

7. Error Handling — Sequence Diagram

Scenario

Invalid request sent to backend.

Flow

1. Angular sends invalid request
2. Controller validation fails
3. Global exception handler triggered
4. Standard error response returned

Key Point

- Consistent error structure across services
-

HOW TO EXPLAIN IN INTERVIEWS

“We use sequence diagrams to show runtime behavior.

Each diagram highlights service boundaries, validation points, and data ownership, ensuring clean microservices communication.”

----- upto here -----

SEQUENCE → CODE CLASS MAPPING

1) User Registration

Sequence Steps

1. User submits registration form
2. API receives request
3. Validate input & business rules

4. Encrypt password
5. Persist user
6. Return response

Code Mapping (User Service)

Step	Layer	Class	Responsibility
1	Angular	RegisterComponent	Collect form data
2	Angular	AuthService	POST /users/register
3	API	UserController	Request mapping
4	DTO	UserRegisterRequest	Bean validation
5	Service	UserService	Uniqueness checks
6	Security	PasswordEncoderConfig	BCrypt encryption
7	Repo	UserRepository	Save user
8	API	UserController	Return response

Key Methods

- `UserController.register(UserRegisterRequest)`
 - `UserService.registerUser(...)`
 - `UserRepository.existsByEmail(...)`
-

2) User Login

Sequence Steps

1. Submit credentials
2. Fetch user
3. Verify password
4. Generate token (optional)
5. Return auth response

Code Mapping

Step	Layer	Class	Responsibility
1	Angular	LoginComponent	Capture credentials
2	Angular	AuthService	POST /users/login
3	API	AuthController	Handle login
4	Repo	UserRepository	Find by email
5	Service	AuthService	Password match
6	Security	JwtTokenProvider	Create JWT
7	API	AuthController	Return token

Key Methods

- `AuthController.login(LoginRequest)`
- `AuthService.authenticate(...)`

3) Product Listing

Sequence Steps

1. UI requests products
2. Fetch from DB
3. Return list

Code Mapping (Product Service)

Step	Layer	Class	Responsibility
1	Angular	ProductListComponent	Load products
2	Angular	ProductApiService	GET /products
3	API	ProductController	Handle request
4	Service	ProductService	Business logic
5	Repo	ProductRepository	Query MongoDB

Key Methods

- `ProductController.getAllProducts()`
 - `ProductService.findAll()`
-

4) Order Placement (Critical Flow)

Sequence Steps

1. Place order
2. Validate request
3. Check stock
4. Reduce inventory
5. Create order
6. Persist & respond

Code Mapping (Order + Product Services)

Step	Service	Class	Responsibility
1	Angular	CheckoutComponent	Submit order
2	Angular	OrderApiService	POST /orders
3	Order API	OrderController	Receive order
4	DTO	OrderRequest	Validate payload
5	Order Svc	OrderService	Orchestrate flow
6	Order Svc	ProductClient	Call Product Service
7	Product API	ProductController	Validate stock
8	Product Svc	InventoryService	Deduct quantity
9	Order Repo	OrderRepository	Save order

Step	Service	Class	Responsibility
10	Order API	OrderController	Return result

Key Methods

- `OrderService.placeOrder (OrderRequest)`
- `ProductClient.checkAndReserveStock (...)`
- `InventoryService.reduceStock (...)`

5) Order Status Update (Admin)

Sequence Steps

1. Admin updates status
2. Validate role
3. Validate transition
4. Update order

Code Mapping

Step	Layer	Class	Responsibility
1	Angular	AdminOrderComponent	Update status
2	Angular	OrderApiService	PUT <code>/orders/{id}/status</code>
3	Security	JwtAuthFilter	Role validation
4	API	OrderController	Accept request
5	Service	OrderService	Validate transition
6	Repo	OrderRepository	Persist status

Key Methods

- `OrderService.updateStatus (orderId, status)`
- `OrderStatusValidator.isValidTransition (...)`

6) Global Validation & Error Handling

Sequence Steps

1. Invalid input
2. Validation fails
3. Standard error response

Code Mapping

Step	Layer	Class	Responsibility
1	DTO	@Valid annotations	Input validation
2	Framework	MethodArgumentNotValidException	Triggered

Step	Layer	Class	Responsibility
3	API	GlobalExceptionHandler	Build error response

Key Classes

- GlobalExceptionHandler
- ApiErrorResponse

7) CI/CD with SonarQube

Sequence Steps

1. Git push
2. Build & tests
3. Sonar scan
4. Quality gate
5. Docker build
6. Deploy

Code/Config Mapping

Step	Tool	File/Class
1	Git	Repository
2	Jenkins	Jenkinsfile
3	Maven	pom.xml
4	SonarQube	sonar-project.properties
5	Docker	Dockerfile
6	Compose	docker-compose.yml

TRACEABILITY (WHY THIS MATTERS)

- **Sequence step** → **Controller** → **Service** → **Repository**
- Every **business rule** is enforced **either in DTO validation or service logic**
- Easy to explain **end-to-end flow** in interviews

How to explain succinctly

“Each sequence diagram step maps directly to a controller endpoint, a service orchestration method, and a repository call. Validation is enforced at DTO and service layers, and failures are handled centrally.”

CODE SKELETON — PACKAGES & CLASSES

1. USER SERVICE (user-service)

```
user-service
├── src/main/java/com/example/user
│   ├── UserServiceApplication.java
│   ├── controller
│   │   └── UserController.java
│   ├── service
│   │   ├── UserService.java
│   │   └── UserServiceImpl.java
│   ├── repository
│   │   └── UserRepository.java
│   ├── model
│   │   └── User.java
│   ├── dto
│   │   ├── UserRegisterRequest.java
│   │   ├── LoginRequest.java
│   │   └── UserResponse.java
│   ├── exception
│   │   ├── UserNotFoundException.java
│   │   ├── DuplicateUserException.java
│   │   └── GlobalExceptionHandler.java
│   └── config
│       └── SecurityConfig.java
```

Key Responsibilities

- UserController → API layer
- UserService → Business rules
- UserRepository → MongoDB access
- SecurityConfig → Password encoding / JWT (optional)

2. PRODUCT SERVICE (product-service)

```
product-service
├── src/main/java/com/example/product
│   ├── ProductServiceApplication.java
│   ├── controller
│   │   └── ProductController.java
│   ├── service
│   │   ├── ProductService.java
│   │   ├── ProductServiceImpl.java
│   │   └── InventoryService.java
│   └── repository
│       └── ProductRepository.java
```

```
├── model
│   └── Product.java
├── dto
│   ├── ProductRequest.java
│   └── ProductResponse.java
└── exception
    ├── ProductNotFoundException.java
    └── GlobalExceptionHandler.java
```

Key Responsibilities

- `InventoryService` → Stock validation & update
 - `ProductServiceImpl` → Core business logic
-

3. ORDER SERVICE (order-service)

```
order-service
├── src/main/java/com/example/order
│   ├── OrderServiceApplication.java
│   ├── controller
│   │   └── OrderController.java
│   ├── service
│   │   ├── OrderService.java
│   │   ├── OrderServiceImpl.java
│   │   └── OrderStatusValidator.java
│   ├── client
│   │   ├── ProductClient.java
│   │   └── UserClient.java
│   ├── repository
│   │   └── OrderRepository.java
│   ├── model
│   │   ├── Order.java
│   │   └── OrderItem.java
│   ├── dto
│   │   ├── OrderRequest.java
│   │   ├── OrderItemRequest.java
│   │   └── OrderResponse.java
│   └── exception
│       ├── OrderNotFoundException.java
│       └── GlobalExceptionHandler.java
```

Key Responsibilities

- `ProductClient` → Calls Product Service
- `OrderStatusValidator` → Valid status transitions
- `OrderServiceImpl` → Orchestrates order flow

4. COMMON / SHARED CONCEPTS (Optional)

```
common
└─ src/main/java/com/example/common
    ├── exception
    │   └─ ApiErrorResponse.java
    ├── util
    │   └─ Constants.java
    └─ config
        └─ SwaggerConfig.java
```

5. ANGULAR FRONTEND (angular-ui)

```
angular-ui
└─ src/app
    ├── core
    │   ├── services
    │   │   ├── auth.service.ts
    │   │   ├── product.service.ts
    │   │   └─ order.service.ts
    │   └─ guards
    │       └─ auth.guard.ts
    ├── modules
    │   ├── auth
    │   │   ├── login.component.ts
    │   │   └─ register.component.ts
    │   ├── product
    │   │   └─ product-list.component.ts
    │   └─ order
    │       ├── checkout.component.ts
    │       └─ order-history.component.ts
    ├── shared
    │   └─ models
    │       ├── user.model.ts
    │       ├── product.model.ts
    │       └─ order.model.ts
    └─ app.module.ts
```

6. TEST STRUCTURE (IMPORTANT)

```
src/test/java
└─ com/example
    ├── controller
    │   └─ UserControllerTest.java
    ├── service
    │   ├── UserServiceTest.java
    │   └─ OrderServiceTest.java
    └─ repository
```

7. DEVOPS & CI/CD FILES

```
capstone-project
├─ Jenkinsfile
├─ docker-compose.yml
├─ user-service/Dockerfile
├─ product-service/Dockerfile
├─ order-service/Dockerfile
├─ angular-ui/Dockerfile
└─ README.md
```

HOW TO EXPLAIN THIS IN INTERVIEWS

“Each microservice follows a layered architecture: Controller → Service → Repository.

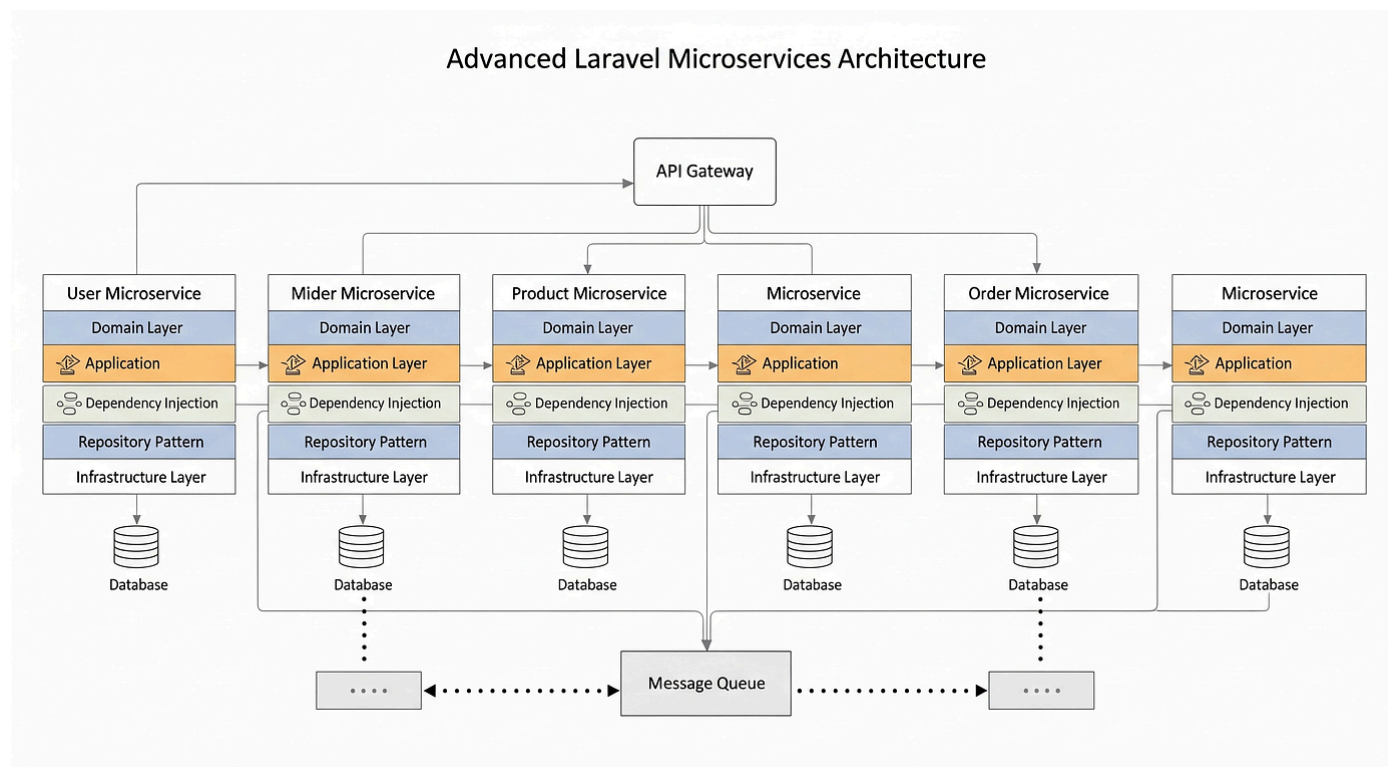
Clients handle inter-service communication.

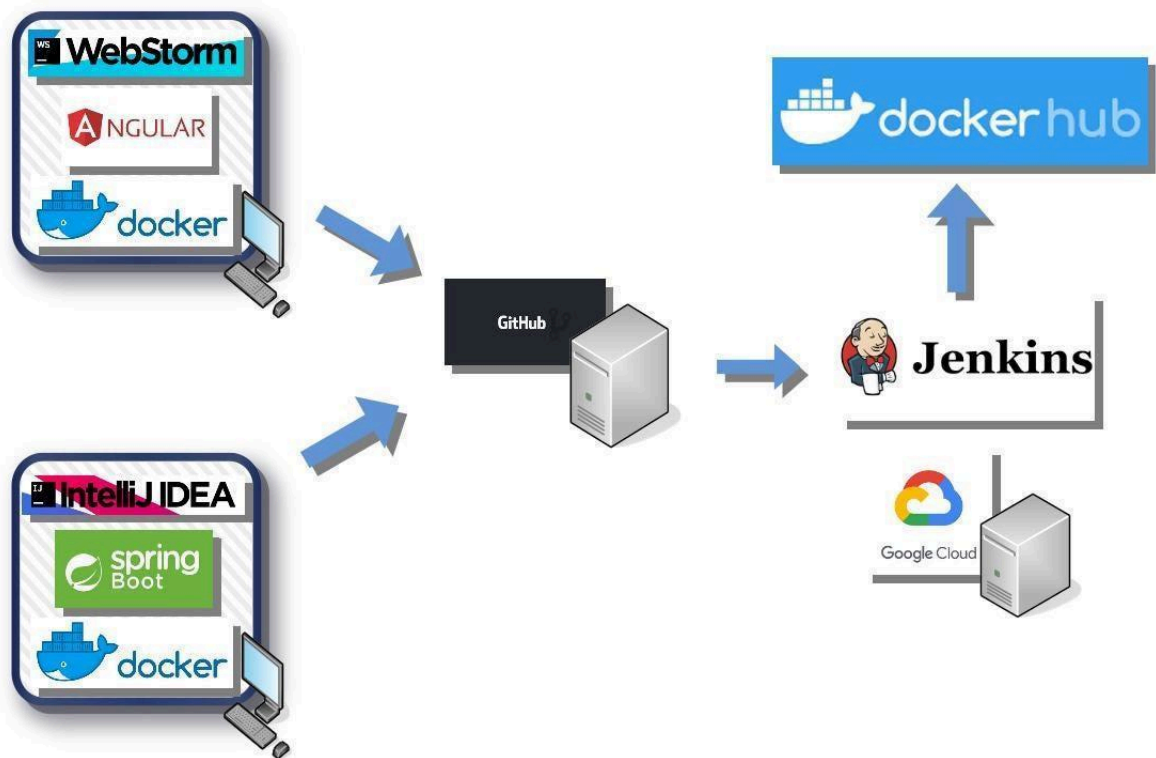
DTOs isolate API contracts.

Business rules live in the service layer.

This makes the system scalable, testable, and maintainable.”

STARTER REPO





1. REPOSITORY STRUCTURE (ROOT)

```
smart-order-management/  
├── user-service/  
├── product-service/  
├── order-service/  
├── angular-ui/  
  
├── common/  
  
├── docker-compose.yml  
├── Jenkinsfile  
├── .gitignore  
└── README.md
```

You can create this repo **as-is** in GitHub.

2. USER SERVICE (Spring Boot)

```
user-service/  
├── src/main/java/com/example/user  
│   ├── UserServiceApplication.java  
│   └── controller  
│       └── UserController.java
```

```
├── service
│   ├── UserService.java
│   └── UserServiceImpl.java
├── repository
│   └── UserRepository.java
├── model
│   └── User.java
├── dto
│   ├── UserRegisterRequest.java
│   ├── LoginRequest.java
│   └── UserResponse.java
├── exception
│   ├── DuplicateUserException.java
│   └── GlobalExceptionHandler.java
├── config
│   └── SecurityConfig.java
├── src/test/java/com/example/user
│   ├── controller/UserControllerTest.java
│   └── service/UserServiceTest.java
├── Dockerfile
├── pom.xml
└── application.yml
```

3. PRODUCT SERVICE

```
product-service/
├── src/main/java/com/example/product
│   ├── ProductServiceApplication.java
│   ├── controller/ProductController.java
│   ├── service
│   │   ├── ProductService.java
│   │   ├── ProductServiceImpl.java
│   │   └── InventoryService.java
│   ├── repository/ProductRepository.java
│   ├── model/Product.java
│   ├── dto/ProductRequest.java
│   └── exception/GlobalExceptionHandler.java
├── src/test/java/com/example/product
│   └── service/ProductServiceTest.java
├── Dockerfile
├── pom.xml
└── application.yml
```

4. ORDER SERVICE

```
order-service/
├── src/main/java/com/example/order
│   ├── OrderServiceApplication.java
│   ├── controller/OrderController.java
│   ├── service
│   │   └── OrderService.java
```

```
|
| | | OrderServiceImpl.java
| | | OrderStatusValidator.java
| | client
| | | ProductClient.java
| | | UserClient.java
| | repository/OrderRepository.java
| | model
| | | Order.java
| | | OrderItem.java
| | dto/OrderRequest.java
|
| src/test/java/com/example/order
| | service/OrderServiceTest.java
|
| Dockerfile
| pom.xml
| application.yml
```

5. ANGULAR FRONTEND

```
angular-ui/
|
| src/app
| |
| | core
| | |
| | | services
| | | |
| | | | auth.service.ts
| | | | product.service.ts
| | | | order.service.ts
| | | guards/auth.guard.ts
| |
| | modules
| | |
| | | auth
| | | |
| | | | login.component.ts
| | | | register.component.ts
| | | product/product-list.component.ts
| | | order/checkout.component.ts
| |
| | shared/models
| | |
| | | user.model.ts
| | | product.model.ts
| | | order.model.ts
|
| Dockerfile
| angular.json
```

6. DOCKER COMPOSE

```
version: '3.8'

services:
  mongodb:
    image: mongo
    ports:
      - "27017:27017"

  user-service:
    build: ./user-service
    ports:
      - "8081:8081"
    depends_on:
      - mongodb
```

```
product-service:
  build: ./product-service
  ports:
    - "8082:8082"
  depends_on:
    - mongodb

order-service:
  build: ./order-service
  ports:
    - "8083:8083"
  depends_on:
    - mongodb

angular-ui:
  build: ./angular-ui
  ports:
    - "4200:80"
```

7. JENKINSFILE (CI/CD + SONARQUBE)

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps { git 'https://github.com/your-org/smart-order-management.git' }
    }

    stage('Build & Test') {
      steps { sh 'mvn clean test' }
    }

    stage('SonarQube Scan') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh 'mvn sonar:sonar'
        }
      }
    }

    stage('Quality Gate') {
      steps {
        timeout(time: 2, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }

    stage('Docker Build & Deploy') {
      steps {
        sh 'docker-compose up -d --build'
      }
    }
  }
}
```

8. README.md (MINIMAL TEMPLATE)

```
# Smart Order Management System

## Tech Stack
- Spring Boot Microservices
- Angular
- MongoDB
- Docker
- Jenkins + SonarQube

## Run Locally
docker-compose up -d

## Services
- User Service: 8081
- Product Service: 8082
- Order Service: 8083
- UI: http://localhost:4200
```

Map **design** → **code structure**

Tell me what you want to generate next.

1. Config Server

- Central configuration

2. Eureka Server

- Service discovery

3. API Gateway

- Routing, JWT validation

4. Auth / User Service

- Login, JWT, roles
- **DB:** PostgreSQL

5. Product Service

- Products & categories
- **DB:** PostgreSQL

6. Warehouse Service

Warehouse + Inventory together

Responsibilities

- Manage warehouses
- Track stock per warehouse
- Stock deduction on dispatch
- Low-stock alerts
- Fulfillment status updates

DB: PostgreSQL

Entities

- Warehouse
- Inventory
- StockMovement
- Fulfillment (or Dispatch)

Order Service

- Order lifecycle
- Stock validation (via Inventory service)
- **DB:** PostgreSQL

Billing / Payment Service

- Invoice & payments
- **DB:** PostgreSQL

Notification Service

- Order updates, alerts